

ארכיטקטורת סוכני AI

LaTeX, RAG, Skills, MCP, API ו-PDF מקצועי ב-LaTeX

בר אילן

קורס AI Agents — מטלה 04

17 ביוני 2026

תקציר

מסמך זה מסכם את ארכיטקטורת סוכני בינה מלאכותית כפי שנלמדה בהרצאה 07 בקורס AI Agents: שלוש שכבות הסוכן (Agent, Skill, Software), מודל שפה גדול וחלון הקשר, RAG, כלים וסקילים, תקשורת בין תוכנות (API, CLI, GUI SDK), פרוטוקול MCP, מגבלות סוכני דפדפן, יצירת PDF באמצעות LuaLaTeX ותבניות CLS, בקרת איכות (QA Skills), ואורכוסטרציה של ריבוי סוכנים. המסמך כולל פרק יישומי קצר על OSINTCTI, וחקירת Ransomware. המקור העיקרי לתוכן הוא תמלול ההרצאה; הביבליוגרפיה משלימה במקורות משניים מאומתים. **מילות מפתח:** סוכן RAGAI, LaTeXMCP, רפטביליות, CTI.

Abstract

This document summarizes agentic AI architecture as taught in Lecture 07 of the AI Agents course: the three-layer model (Software, Skill, Agent), large language models and context windows, retrieval-augmented generation, tools and skills, inter-application communication (SDK, GUI, CLI, API), the Model Context Protocol, limitations of browser-based GUI agents, professional PDF production with LuaLaTeX and custom CLS templates, QA skills, and multi-agent orchestration. A short applied chapter covers CTI, OSINT, and ransomware investigation workflows. Primary source: lecture transcript; bibliography supplements with verified references. **Keywords:** AI agent, RAG, MCP, LaTeX, reproducibility, CTI.

תוכן העניינים

1	תקציר	1
1	Abstract	1
5	מבוא לסוכני AI	1
5	1.1 מהו סוכן AI?	
5	1.2 מדוע ארכיטקטורה חשובה?	
5	1.3 מבנה המסמך	
5	1.4 מטרות למידה	
6	1.5 הקשר אקדמי	
6	2 שלוש שכבות של סוכן: Software, Skill, Agent	
6	2.1 שכבת Software — התשתית הקבועה	
6	2.2 שכבת Skill — שפה טבעית מעל קוד	
6	2.3 שכבת Agent — המוח וההחלטות	
7	2.4 זרימת עבודה אופיינית	
7	2.5 יישום ארגוני	
7	3 LLM, Context Window וזיכרון	
7	3.1 מהו LLM?	
8	3.2 Context Window — זיכרון העבודה	
8	3.3 Memory — סוגי זיכרון	

8	הרכבת פרומפט מלא	3.4
8	אסטרטגיות לניהול הקשר	3.5
9	השלכות על עלות	3.6
9	RAG: Retrieval-Augmented Generation	4
9	עקרון הפעולה	4.1
9	מדוע לא לאמן מחדש?	4.2
10	דוגמת זרימה (פסאודו-קוד)	4.3
10	רכיבי מערכת RAG	4.4
10	קשר לקורס	4.5
10	מגבלות	4.6
11	Agent Skills ו-Tools, Skills	5
11	Tools — כלי ביצוע	5.1
11	Skills — קבצי הנחיה	5.2
11	Agent Skills — סקילים ברמת הסוכן	5.3
11	תהליך בחירת Skill	5.4
12	דוגמה: Skill לקומפילציית LaTeX	5.5
12	Skills מול Tools — סיכום	5.6
12	תקשורת בין תוכנות: SDK, GUI, CLI, API	6
13	שכבת הפונקציות	6.1
13	SDK — ערכת פיתוח	6.2
13	GUI — ממשק גרפי	6.3
13	CLI — ממשק שורת פקודה	6.4
13	API — חשיפה לעולם החיצון	6.5
14	Skill כעטיפה ל-API	6.6
14	כיוון עתידי	6.7
14	MCP: Model Context Protocol	7
15	מהו MCP?	7.1
15	MCP מול API ישיר	7.2
15	מודל שרת-לקוח	7.3
15	דוגמת תצורה (מושגית)	7.4
15	MCP בתוך ארכיטקטורת הסוכן	7.5
16	מגבלות ואבטחה	7.6
16	קשר לפרויקט הנוכחי	7.7
16	מגבלות עבודה מול GUI וסוכני דפדפן	8
17	כיצד פועל סוכן דפדפן?	8.1
17	חסרונות שזוהו בשיעור	8.2
18	מדוע GUI מפריע לעידן הסוכנים?	8.3
18	השוואה מעשית	8.4
18	מתי בכל זאת להשתמש ב-GUI Agent?	8.5
18	סיכום פרק	8.6

18	9 LaTeX, LuaLaTeX ויצירת PDF מקצועי
19	9.1 מדוע LaTeX?
19	9.2 סוגי קבצי מסמך
19	9.3 LuaLaTeX – הקומפיילר
20	9.4 מבנה פרויקט לסוכן
20	9.5 יתרונות לעבודה עם סוכן AI
20	9.6 מגבלות
20	10 CLS Templates ו-Reproducibility
20	10.1 מהו קובץ CLS?
21	10.2 CLS כ-Tool
21	10.3 לולאת תיקון רפטבילית
21	10.4 יישום ארגוני
21	10.5 קשר למטלה 04
22	11 QA Skills ובקרת איכות
22	11.1 פילוסופיית QA
22	11.2 מבנה היררכי
23	11.3 Fixer ו-Detector
23	11.4 דוגמת Skill QA-Table
23	11.5 יתרונות מערכת QA
23	11.6 יישום במטלה 04
24	12 Agent Orchestration: סוכן יחיד מול ריבוי סוכנים
24	12.1 סוכן יחיד, ריבוי קובעים
24	12.2 ריבוי סוכנים, קובע אחד לכל אחד
24	12.3 חסרונות ריבוי סוכנים
25	12.4 תפקיד האורכוסטרטור
25	12.5 המלצה מעשית
25	13 יישום בעולם הסייבר: CTI, OSINT וחקירת אירועי Ransomware
26	13.1 הקשר: Ransomware ו-CTI
26	13.2 ארכיטקטורת סוכן לחקירה
26	13.3 תרחיש לדוגמה
26	13.4 דוגמת RAG ל-CTI
27	13.5 שיקולי אבטחה ואתיקה
27	13.6 מגבלות ההדגמה
27	14 סיכום אינטגרטיבי: המשמעות הניהולית והטכנולוגית
27	14.1 מעבר מ-GUI ל-API
27	14.2 שלוש שכבות בארגון
28	14.3 מודל עלויות
28	14.4 רפטביליות כיתרון תחרותי
28	14.5 סיכונים וניהול סיכונים
29	14.6 קשר לתפקיד מומחה AI בארגון

29	15 סיכום ומסקנות
29	15.1 ממצאים מרכזיים
29	15.2 תרומת מטלה 04
30	15.3 כיווני המשך
30	15.4 מילה אחרונה

רשימת האיורים

6	1	שלוש שכבות הסוכן לפי חומר ההרצאה: תוכנה, סקיל וסוכן
9	2	זרימת RAG: מהפרומפט לשליפה וקטורית, הזרקה לחלון הקשר ותשובת המודל
12	3	מחסנית תקשורת בין תוכנות: פונקציות, API/CLI, GUI SDK
14	4	השוואה מושגית: סוכן יכול לגשת ליכולות תוכנה דרך APISkill, או MCP
17	5	סוכן דפדפן מול סוכן טקסט/API: עלות, מורכבות וזמן (אילוסטרציה לפי ההרצאה)
22	6	היררכיית QA Skills לפי דוגמת המרצה: מנהל-על ומומחי תחום

רשימת הטבלאות

7	1	השוואת שלוש השכבות
10	2	רכיבי מערכת RAG טיפוסית
12	3	הבחנה בין Tool ל-Skill
14	4	השוואת דרכי גישה לתוכנה
16	5	השוואה: MCP + APISkill מול MCP
18	6	סוכן GUI מול סוכן API/טקסט
21	7	Word/LaTeX מול Word לרפטיביות
23	8	רכיבי מערכת QA למסמכי PDF
25	9	השוואת מודלי אורכוסטרציה
26	10	מיפוי תפקידי סוכן בחקירת Ransomware
28	11	מסגרת החלטה ארגונית לסוכן AI

1 מבוא לסוכני AI

בעשור האחרון עברה הבינה המלאכותית ממודלים של סיווג וחיזוי סטטיסטי, אל מערכות שמסוגלות לנהל שיחה, לתכנן פעולות ולבצע משימות מורכבות בסביבה דיגיטלית. במסגרת קורס AI Agents נלמד כיצד לבנות, להפעיל ולנהל סוכנים — ישויות תוכנה שמשלבות מודל שפה גדול (LLM) עם זיכרון עבודה, כלים חיצוניים ויכולת תכנון. מסמך זה מסכם את הארכיטקטורה כפי שנלמדה בהרצאה 07, ומרחיב אותה לכיוון יישומי בסייבר.

1.1 מהו סוכן AI?

סוכן AI הוא מערכת שמקבלת מטרה בשפה טבעית, מפרקת אותה לצעדים, בוחרת כלים מתאימים ומחזירה תוצאה. בניגוד לצ'אטבוט קלאסי, הסוכן `llm` הוא יכול לקרוא קבצים, להפעיל API, לחפש במאגר ידע או לקמפל מסמך PDF -ל- LaTeX . לפי חומר ההרצאה, הסוכן אינו עומד בפני עצמו — הוא נשען על שכבות תחתונות של תוכנה קבועה ושכבת Skill שמעטפת את הקוד בשפה טבעית [1].

תובנה

המעבר ממודל שפה לסוכן פעיל דורש שלושה מרכיבים: **תכנון** (מה לעשות), **ביצוע** (כלים) ו**זיכרון** (הקשר + אחסון חיצוני). ללא אחד מהם, המערכת נשארת מחולל טקסט בלבד.

1.2 מדוע ארכיטקטורה חשובה?

ארגונים שמטמיעים סוכני AI נתקלים במהירות בבעיות חוזרות: עלויות טוקנים גבוהות, תוצאות לא עקביות, קושי בבקרת איכות ותלות ב-GUI במקום ב-API. הבנת הארכיטקטורה — שלוש השכבות, MCPRAG, תבניות CLS ורב-סוכנים — מאפשרת לבנות מערכות **רפטביליות** (reproducible) שחוזרות על אותה איכות בכל הרצה.

1.3 מבנה המסמך

המסמך בנוי מ-15 פרקים: מיסודות הסוכן ושכבותיו, דרך מנגנוני ידע ותקשורת, ועד יצירת PDF מקצועי, בקרת איכות ואורכוסטרציה. פרק 13 מדגים יישום קצר בתחום OSINTCTI/OSINTCTI וחקירת Ran-somware. הפרקים האחרונים מסכמים את המשמעות הניהולית והטכנולוגית.

1.4 מטרות למידה

לאחר קריאת המסמך, הקורא אמור להבין:

- את ההבדל בין שכבת תוכנה, Skill וסוכן.
- כיצד RAG מרחיב את חלון ההקשר מבלי לאמן מחדש את המודל.
- מדוע MCP שונה מ-Skill סטטי עם API.
- מדוע עבודה טקסטואלית/API עדיפה על סוכן דפדפן לייצור מסמכים.
- כיצד תבנית CLS וסקילי QA מבטיחים רפטביליות.

הערה

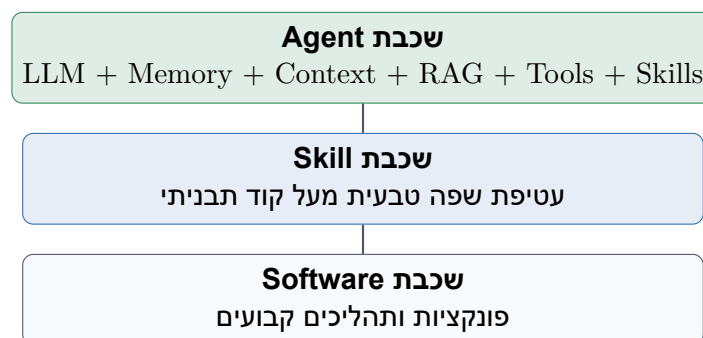
מונחים באנגלית מסומנים בפקודה `\eng{}` — לדוגמה: Retrieval-Context Window, Augmented Generation.

1.5 הקשר אקדמי

מטלה 04 בקורס AI Agents דורשת הפקת מסמך PDF מקצועי באמצעות LuaLaTeX וקובץ CLS מותאם. המסמך משקף את יכולת הסטודנט להגדיר הנחיות לסוכן, לבנות תבנית עיצוב נפרדת ולייצר תוכן אקדמי מלא — לא שלד ריק. המקור העיקרי לתוכן הוא תמלול הרצאה 07 (sources/transcript-lecture07.txt).

2 שלוש שכבות של סוכן: Software, Skill, Agent

אחת התרומות המרכזיות של ההרצאה היא מודל השכבות השלוש, המסביר כיצד סוכן AI בנוי מרכיבים שונים בעלי אחריות ברורה. המודל מאפשר להבין היכן להציב קבועה, היכן להשתמש בשפה טבעית ואיפה "המוח" של הסוכן פועל.



איור 1: שכבת Agent, שכבת Skill, שכבת Software

2.1 שכבת Software — התשתית הקבועה

בשכבה התחתונה נמצאת התוכנה התבניתית: פונקציות, אלגוריתמים ותהליכים שחייבים לרוץ באופן דטרמיניסטי. דוגמאות: חישוב מרחק קוסינוס בין וקטורים, קריאת קובץ CSV, שליחת בקשת HTTP או קומפילציה של LaTeX ל-PDF. שכבה זו אינה "מבינה" שפה — היא מבצעת הוראות מדויקות.

2.2 שכבת Skill — שפה טבעית מעל קוד

שכבת ה-Skill עוטפת את התוכנה במעטפת של שפה טבעית. במקום לספק פרמטרים מדויקים לפונקציה, המשתמש או הסוכן מתארים בכוונה כללית מה נדרש, וה-LLM בשיתוף ה-Skill ממפה את התיאור לפרמטרים הנכונים. לדוגמה: "חפש דירות זולות ברחוב רוטשילד" במקום קואורדינטות מדויקות של שדות בטופס.

תובנה

Skill הוא לא "סוכן" — הוא מנגנון שמאפשר לדבר עם תוכנה קיימת בשפה חופשית. הסוכן משתמש ב-Skills כאשר הוא מזהה שמשימה מתאימה לתיאור של אחד מהם.

2.3 שכבת Agent — המוח וההחלטות

שכבת הסוכן כוללת את ה-LLM, את חלון ההקשר (Context Window), את הזיכרון, את ה-RAG, את הכלים (Tools) ואת היכולת לבחור Skills מתאימים. הסוכן מקבל מטרה, מתכנן, מפעיל כלים ומסכם תוצאות. בניגוד ל-Skill בודד, הסוכן יכול לשרשר מספר פעולות ולהתאים את עצמו למשוב.

טבלה 1: מיון מיון מיון

מאפיין	Software	Skill	Agent
שפה טבעית	לא	כן (ממשק)	כן (ליבה)
קביעות	גבוהה	בינונית	נמוכה
רב-שלבי	לא	מוגבל	כן
דוגמה	cosine()	תרגום מסמך	חקירת אירוע

2.4 זרימת עבודה אופיינית

כאשר משתמש שואל "מה בירת מדינת ישראל?", הסוכן:

- 1. מנתח את הפרומפט ב-LLM.
- 2. מחפש Skill או כלי חיפוש רלוונטי.
- 3. מפעיל את שכבת ה-Software (למשל חיפוש ב-RAG).
- 4. מרכיב תשובה מתוך חלון ההקשר ומחזיר למשתמש.

אזהרה

בלבול בין שכבות — למשל הצבת לוגיקה עסקית קריטית רק בפרומפט ללא קוד תבניתי — עלול לגרום לתוצאות לא עקביות. כלל אצבע: מה שחייב להיות זהה בכל הרצה שייך לשכבת Software או ל-CLS.

2.5 יישום ארגוני

בארגון, ניתן לייצר ספריית Skills לכל מחלקה: כספים, משאבי אנוש, CTI. הסוכן המרכזי בוחר את ה-Skill המתאים לפי תיאור בראש הקובץ. כך נשמרת אחידות בלי לשכפל קוד בכל סוכן.

3 LLM, Context Window וזיכרון

מודל השפה הגדול (LLM) הוא ליבת הסוכן, אך חשוב להבין את מגבלותיו: למודל אין זיכרון פנימי מתמשך בין הרצות, וכל המידע שהוא "רואה" מוגבל לגודל חלון ההקשר (Context Window) [1].

3.1 מהו LLM?

LLM (Large Language Model) הוא רשת נוירונים מאומנת על כמויות עצומות של טקסט, המסוגלת לחזות המשך סביר לרצף מילים. בפועל, המודל מבצע משימות כמו סיכום, תרגום, כתיבת קוד ותכנון — אך תמיד בתוך מסגרת הטוקנים שהוזנו אליו בבקשה הנוכחית.

3.2 Context Window — זיכרון העבודה

חלון ההקשר הוא "שולחן העבודה" של הסוכן בזמן שיחה. בו מתנהלים:

System Prompt — ההנחיות הכלליות לסוכן.

- היסטוריית השיחה — שאלות ותשובות קודמות.
- תוצאות כלים — פלט מחיפוש, הרצת קוד, שליפת RAG.
- תוכן Skills שנבחרו לבקשה הנוכחית.

כאשר החלון מתמלא, יש לקצץ, לסכם או להעביר מידע לאחסון חיצוני. זו אחת הסיבות לחשיבות RAG ולניהול זיכרון ארוך טווח.

הערה

לפי ההרצאה: "אין לו זיכרון בעצמו — הכל נמצא אצלנו במחשב וכל פעם אני טוען את ה-Context Window מחדש עם כל מה שהיה עד עכשיו."

3.3 Memory — סוגי זיכרון

נהוג להבחין בין:

זיכרון קצר טווח

תוכן חלון ההקשר הנוכחי.

זיכרון ארוך טווח

מאגר חיצוני — קבצים, מסד נתונים, vector store.

זיכרון פרוצדורלי

Skills, תבניות CLS וכללים קבועים.

3.4 הרכבת פרומפט מלא

לפני כל קריאה ל-LLM, המערכת מרכיבה חבילה:

$$\text{Input} = \text{System} + \text{Skills} + \text{RAG Docs} + \text{User Prompt} + \text{Tool Outputs}$$

המודל מייצר תשובה; התשובה מתווספת לחלון ההקשר; המחזור חוזר.

3.5 אסטרטגיות לניהול הקשר

1. **סיכום תקופתי** — סוכן מסכם שיחה ארוכה ומחליף היסטוריה בסיכום.
2. **שליפה סלקטיבית** — רק מסמכים רלוונטיים מ-RAG נכנסים לחלון.
3. **חלוקה למשימות** — אורכוסטרטור מעביר תת-משימות לסוכנים עם הקשר מצומצם.

תובנה

Chain-of-Thought [2] משפר ביצועים על ידי הרחבת הטקסט הביניים בחלון ההקשר — אך מגדיל צריכת טוקנים. איזון בין איכות לעלות הוא החלטת ארכיטקטורה.

3.6 השלכות על עלות

כל טוקן בחלון ההקשר — קלט ופלט — עולה כסף במודלים מסחריים. שליפת מסמכים מיותרים ל-RAG, צילומי מסך מדפדפן והיסטוריה ארוכה מנפחים את העלות. לכן מעבר מ-API ל-GUI/טקסט הוא גם החלטה כלכלית.

4 RAG: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) הוא מנגנון שמאפשר לסוכן לגשת לידע חיצוני מבלי לאמן מחדש את המודל [3]. בהרצאה הושווה ה-RAG ל"דיסק קשיח": מאגר גדול המסודר וקטורית לחיפוש סמנטי.



איור 2: RAG: חיפוש סמנטי, מאגר גדול המסודר וקטורית לחיפוש סמנטי.

4.1 עקרון הפעולה

כאשר משתמש שואל שאלה:

1. הפרומפט מומר לוקטור (הטמעה — embedding).
 2. מתבצע חיפוש במאגר הוקטורים באמצעות מרחק קוסינוס (או מטריקה דומה).
 3. המסמכים הקרובים ביותר סמנטית נשלפים מהמאגר.
 4. המסמכים מצורפים לחלון ההקשר יחד עם הפרומפט.
 5. ה-LLM מייצר תשובה המבוססת על המידע שנשלף.
- דוגמה מההרצאה: שאלה על בירת ישראל → חיפוש בוויקיפדיה ובמסמכים רלוונטיים → הזרקה ל-Context Window → תשובה מדויקת.

4.2 מדוע לא לאמן מחדש?

אימון מחדש של LLM על מאגר ארגוני הוא יקר, איטי ומסובך לתחזוקה. RAG מאפשר:

- עדכון ידע בזמן אמת (הוספת מסמכים למאגר).
- ציטוט מקורות (איזה מסמך שימש בתשובה).
- שליטה בהרשאות (מאגרים נפרדים לפי מחלקה).

4.3 דוגמת זרימה (פסאודו-קוד)

Listing 1: Pipeline בסיסי של RAG

```
def rag_answer (llm , vector_db , question) 1
    (question)embed = q_vec 2
    (5=top_k , q_vec)search.vector_db = docs 3
    (docs)join ."n\" = context 4
    "{question} : nQuestion\n\n{context}\n: Context" f = prompt 5
    (prompt)generate .llm return 6
```

אזהרה

RAG אינו מבטיח אמיתות — אם המאגר מכיל מידע שגוי, המודל עלול להמחיש אותו בביטחון. נדרשת בקרת איכות על המקורות ואימות תשובות.

4.4 רכיבי מערכת RAG

טבלה 2: רכיבי מערכת RAG

רכיב	תפקיד	דוגמה
Ingestion	פיצול ואינדוקס מסמכים	של 512 טוקנים chunk
Embedding	המרה לוקטור	text-embedding-3-small
Vector DB	אחסון וחיפוש	ChromaPinecone ,
Retriever	בחירת קטעים	top-k , דמיון
Generator	תשובה סופית	GPT-4Claude ,

4.5 קשר לקורס

במטלות קודמות בקורס (למשל תרגום רב-שלבי) נעשה שימוש בהשוואת וקטורים למדידת סטייה סמנטית. RAG משתמש באותו עקרון — מרחק וקטורי — למטרה אחרת: שליפת ידע במקום הערכת דמיון בין תרגומים.

4.6 מגבלות

- איכות התלויה בגודל ואיכות ה-chunk.
- חיפוש שגוי אם השאלה מנוסחת רחוק מהמסמכים.
- עלות הטמעה ואחסון למאגרים גדולים.
- צורך באבטחת מידע — דליפת מסמכים רגישים לפרומפט.

תובנה

בפרק 13 נראה כיצד RAG יכול לתמוך בחקירת Ransomware על ידי אינדוקס דוחות CTI, הערות כופר ו-IOCs היסטוריים.

5 Agent Skills-ו Tools, Skills

סוכן AI מבדיל בין יכולת ביצוע (Tool) לבין מעטפת ידע והנחיה (Skill). הבנה נכונה של ההבחנה מאפשרת לבנות מערכות מודולריות שניתן לתחזק ולהרחיב [4, 5].

5.1 Tools — כלי ביצוע

Tools הם התוכנות והפונקציות שהסוכן מפעיל: חיפוש באינטרנט, הרצת Python, קריאת קובץ, קומפילציית $\text{\LaTeX}$, שליחת HTTP. ה-LLM לא "מריץ" קוד בעצמו — הוא מחליט `"""` כלי להפעיל ועם אילו פרמטרים, והסביבה מבצעת בפועל.

5.2 Skills — קבצי הנחיה

Skill הוא קובץ (לרוב Markdown) המגדיר תפקיד, כללים ודוגמאות. בתחילת כל Skill מופיע `descrip-` tion שמסביר מתי להשתמש בו. הסוכן סורק את רשימת ה-Skills, בוחר את הרלוונטיים לפרומפט וטוען את תוכנם לחלון ההקשר — בדומה לבחירת מומחה מספרייה.

הערה

במטלה 02 בקורס נבנו Skills נפרדים לכל סוכן תרגום (`skills/en_to_fr.md` וכו'). אותו עיקרון חל על ייצור QAPDF, וחקירת סייבר.

5.3 Agent Skills — סקילים ברמת הסוכן

Agent Skills (כמו ב-Cursor או Claude) הם מפרטים שניתן לשתף בין פרויקטים: הוראות מערכת, תבניות פרומפט וכללי בטיחות. הם מהווים Memory פרודורלי — ידע שתמיד זמין לסוכן בלי לאחסן אותו בכל שיחה.

5.4 Skill תהליך בחירת Skill

לפי ההרצאה, כשמשתמש שואל שאלה:

1. ה-LLM סורק תיאורי Skills זמינים.
2. נבחרים Skills עם התאמה סמנטית (למשל "חיפוש באינטרנט").
3. תוכן ה-Skill נטען ל-Context Window.
4. הסוכן מפעיל Tools שה-Skill מגדיר או מרמז עליהם.

5.5 דוגמה: Skill לקומפילציית LaTeX

Listing 2: קטע מתוך Skill לייצור PDF

```
1 Skill Compile LaTeX #
2 :document PDF a requests user the When
3 .1 (tex.main in styles inline not do) cls.template-ai-agentic Use
4 .2 x2 lualatex <- bibtex <- lualatex :with Compile
5 .3 document-per not , file cls. the in issues RTL recurring Fix
6 .4 delivery before bibliography and TOC Verify
```

5.6 Skills מול Tools — סיכום

טבלה 3: Skill ו-Tool

	Tool	Skill
מהות מיקום	קוד רץ	הנחיות טקסט skills/*.md
שינוי דוגמה	src/MCP , מפתח תוכנה	מומחה תחום
	vector_compare.py	QA-Table.md

תובנה
ReAct [4] משלב חשיבה (Reasoning) ופעולה (Acting) — המודל כותב מחשבות, בוחר כלי, מקבל תוצאה וחוזר חלילה. זהו הבסיס לרב-סוכנים מודרניים.

6 תקשורת בין תוכנות: SDK, GUI, CLI, API

תוכנות מודרניות נבנות בשכבות. הבנת השכבות חיונית לתכנון סוכנים שצריכים לדבר עם מערכות קיימות — בין אם דרך ממשק אנושי או ישירות דרך ממשק מכונה.



איור 3: שכבות תקשורת בין תוכנות: API, CLI, GUI, SDK

6.1 שכבת הפונקציות

בתחתית נמצאות פונקציות התוכנה: חישוב, חיפוש, כתיבה לדיסק. שכבה זו אינה נגישה ישירות למשתמש הקצה — היא ליבת הלוגיקה.

6.2 SDK — ערכת פיתוח

SDK (Software Development Kit) עוטף את הפונקציות בממשק נוח למפתחים. כל פעולה חיצונית בתוכנה אמורה לעבור דרך ה-SDK. כשבונים GUI או CLI, הם קוראים ל-SDK ולא ישירות לפונקציות גולמיות.

6.3 GUI — ממשק גרפי

GUI (Graphical User Interface) מיועד לבני אדם: תפריטים, חלונות, גרירה בעכבר. דוגמאות: PowerPoint, Excel, דפדפן. סוכן שעובד דרך GUI מדמה משתמש אנושי — לוחץ, מצלם מסך, מקליד — וזה יקר בטוקנים ובזמן.

6.4 CLI — ממשק שורת פקודה

CLI (Command Line Interface) הוא תפריט טקסטואלי בטרמינל: לחיצה על A מבצעת פעולה אחת, B — אחרת. פשוט, מהיר וידידותי לסוכנים. ההרצאה מדגישה מעבר מעבודה ב-GUI לעבודה בטרמינל ובטקסט.

6.5 API — חשיפה לעולם החיצון

API (Application Programming Interface) מאפשר לתוכנה חיצונית לבצע פעולות מורשות. כשתוכנה א' רוצה לדבר עם תוכנה ב', הן מתקשרות דרך API — בתנאי שיודעים לתכנת את החיבור.

Listing 3: דוגמת בקשת API (JSON)

```

1 1.1/HTTP compile/documents/v1/api/ POST
2 json/application :Type-Content
3
4 }
5 , "tex.main" : "source"
6 , "lualatex" : "engine"
7 3 : "runs"
8 {

```

טבלה 4: סכומים מסומנים בסמנים

שיטה	קהל יעד	יתרון	חסרון לסוכן
GUI	אדם	אינטואיטיבי	טוקנים, איטיות
CLI	מפתח/סוכן	מהיר, טקסט	דורש למידה
API	מערכות	אוטומציה מלאה	דורש תיעוד
SDK	מפתח פנימי	שליטה מלאה	לא חשוף תמיד

6.6 Skill כעטיפה ל-API

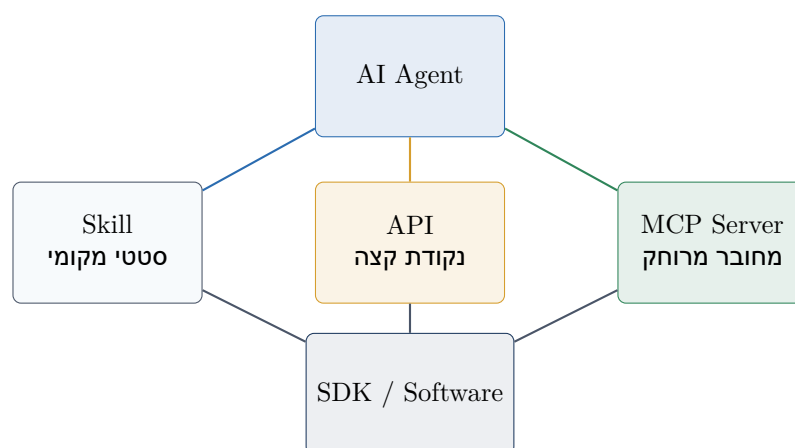
כפי שנלמד בהרצאה, ניתן לעטוף API ב-Skill ולאפשר "AI via API" — שפה חופשית שמתורגמת לקריאות מובנות. כך תוכנה א' ותוכנה ב' יכולות לתקשר דרך סוכנים בלי שהמפתח יזכור כל פרמטר.

6.7 כיוון עתידי

החזון מההרצאה: מערכות הפעלה מבוססות AI שבהן המשתמש מדבר בשפה חופשית והמערכת בוחרת כלים (PythonExcel, דפדפן) בשקיפות. למימוש חזון זה נדרש שרוב התוכנות יחשפו יכולות בטקסט/API, לא רק ב-GUI.

7 MCP: Model Context Protocol

MCP (Model Context Protocol) הוא תקן שמגדיר כיצד סוכני AI מתחברים לשירותים חיצוניים — מסדי נתונים, מאגרי קבצים, כלי פיתוח ועוד [6]. ההרצאה מציגה את MCP כאלטרנטיבה מודרנית לעבודה ישירה עם API ו-Skills סטטיים.



איור 4: סכומים מסומנים בסמנים: Skill API, MCP

7.1 מהו MCP?

MCP הוא "שרת" שרץ בנקודת קצה — במחשב המקומי או אצל ספק התוכנה — ומציע לסוכן רשימת כלים (tools), משאבים (resources) ותבניות פרומפט. הסוכן "מכיר" את ה-MCP הרשום אצלו (בדומה לרשימת Skills) ומפעיל אותו כשהמשימה מתאימה.

7.2 MCP מול API ישיר

בעבודה מול API ישיר, המפתח חייב לדעת כתובת, אימות, סכמת בקשות ועדכוני גרסה. ב-MCP, ספק התוכנה מטפל בשכבה זו; הסוכן מגלה יכולות דינמית מהשרת.

תובנה

ההבדל המרכזי מההרצאה: Skill סטטי דורש התקנה מחדש כשה-API משתנה; MCP מחובר "אוניין" ומתעדכן אוטומטית מול היצרן.

7.3 מודל שרת-לקוח

MCP מבוסס על מודל שרת ולקוח, בדומה לגלישה באינטרנט: הסוכן "גולש" לשרת ומבצע פעולות מורשות. אין צורך לדעת כל פרט טכני על מיקום התוכנה — רק את נקודת הקצה של ה-MCP.

7.4 דוגמת תצורה (מושגית)

Listing 4: דוגמת הגדרת MCP בקובץ JSON

```
1  }
2  } : "mcpServers"
3  } : "filesystem"
4  , "npx" : "command"
5  , "filesystem-server / modelcontextprotocol@" : "args"
6  ["docs / ."
7  , {
8  } : "browser"
9  , "browser-ide-cursor" : "command"
10  [] : "args"
11  {
12  {
13  {
```

7.5 MCP בתוך ארכיטקטורת הסוכן

כפי שהוסבר: הסוכן מכיל פנייה ל-MCP. כמו חיפוש Skills, הוא בוחר את השרת המתאים. בתוך ה-MCP עדיין קיים API שמדבר עם ה-SDK — אך הסוכן לא חשוף לפרטים אלה.

טבלה 5: `MCP` + `APISkill`

מאפיין	Skill + API	MCP
מיקום	מקומי, סטטי	שרת מרוחק/מקומי
עדכון	ידני	אוטומטי
גילוי יכולות	Skill מתוך קובץ	מתוך שרת
בין סוכנים	שפה חופשית תקשורת	שפה חופשית

7.6 מגבלות ואבטחה

- הרשאות — אילו כלי MCP מורשים לסוכן?
- אימות — מפתחות API וסודות לא בפרומפט.
- זמינות — תלות ברשת ובשרת חיצוני.
- תאימות — לא כל מוצר תומך ב-MCP עדיין.

אזהרה

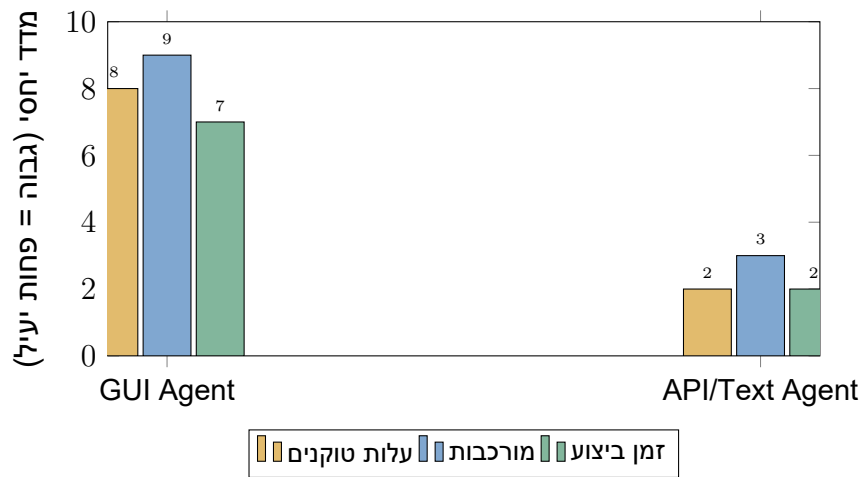
חשיפת MCP עם גישה לקבצים מערכת או דפדפן דורשת הגדרת הרשאות זהירה — סוכן שגוי עלול למחוק קבצים או לשלוח מידע רגיש.

7.7 קשר לפרויקט הנוכחי

ב-Cursor, שרתי MCP כמו cursor-ide-browser ו-playwright מאפשרים לסוכן לפעול בסביבת הפיתוח. מטלה 04 עצמה מדגימה עבודה טקסטואלית (LaTeX) — גישה עדיפה על סימולציית GUI לעריכת מסמכים.

8 מגבלות עבודה מול GUI וסוכני דפדפן

ההרצאה מציגה הדגמה של סוכן דפדפן (Claude עם תוסף, Nano Browser) שמבצע חיפוש ביד2 — ומדגישה את המחיר הכבד של גישה זו. פרק זה מסכם חסרונות, אלטרנטיבות וכיוון המעבר לעבודה טקסטואלית.



איור 5: השוואה בין שני סוגי אגנטים: GUI ו-API/Text. המידות הנמדדות הן זמן ביצוע, מורכבות ומחיר טוקנים.

8.1 כיצד פועל סוכן דפדפן?

הסוכן:

1. מקבל מטרה בשפה טבעית ("חפש דירות ברחוב רוטשילד").
 2. בונה תוכנית עבודה ומבקש אישור.
 3. פותח דפדפן, מצלם מסך (screenshot), מנתח תמונה.
 4. מקליד בשדות, לוחץ כפתורים, חוזר עד להשלמה.
- כל צילום מסך וניתוח תמונה צורכים טוקנים רבים. כל טעות UI (כיוון כתיבה עברית, חלון מודאלי) מחייבת ניסיון חוזר.

8.2 חסרונות שזוהו בשיעור

סטודנטים והמרצה ציינו:

- **זמן** — משימה פשוטה נמשכת דקות ארוכות.
- **עלות** — תשלום לפי טוקנים; צילומי מסך יקרים במיוחד.
- **אבטחה** — התחברות לגימייל, סיסמאות, אישורי משתמש.
- **RTL** — אתרים שכותבים משמאל לימין שוברים הקלדה בעברית.
- **שבירות** — שינוי בעיצוב האתר שובר את הסוכן.

הערה

המרצה הזכיר כלי "הפוך על הפוך" להחלפת כיוון מקלדת עברית-אנגלית — פתרון אנקדוטלי, לא ארכיטקטוני.

8.3 מדוע GUI מפריע לעידן הסוכנים?

החזון מההרצאה: מערכת הפעלה שמבינה שפה חופשית לא צריכה לחקות לחיצות עכבר. GUI תוכנן לבני אדם — תפריטים, אייקונים, משוב ויזואלי. סוכן שעובד דרך GUI מבזבז משאבים על סימולציה אנושית במקום לגשת ישירות ל-API או לטקסט.

8.4 השוואה מעשית

טבלה 6: GUI ו-API

סוכן/API	טקסט	סוכן דפדפן	מדד
JSON, קוד, טקסט	תמונות מסך	קלט לסוכן	
נמוכה	גבוהה	עלות טיפוסית	
מהירה	איטית	מהירות	
יציבה יותר	שבירה בעיצוב	אמינות	
LuaLaTeXCLS +	Word GUI-ב עריכת	דוגמה במטלה 04	

8.5 מתי בכל זאת להשתמש ב-GUI Agent?

יש מקרים שבהם אין API:

- אתרים ישנים ללא ממשק מכונה.
 - טפסים ממשלתיים חד-פעמיים.
 - משימות חד-פעמיות שלא שוות פיתוח API.
- אך עבור תהליכים רפטוביליים — דוחות, הצעות מחיר, מסמכי קורס — יש להעדיף LaTeX, Markdown ו-API.

תובנה

המעבר מ-GUI לטקסט הוא לא רק טכני — הוא ארגוני: חברות שמחשיפות API ומכפ MCP מאפשרות אוטומציה; אלה שנשענות רק על ממשק גרפי נשארות מחוץ למערכת הסוכנים.

8.6 סיכום פרק

סוכן דפדפן הוא כלי עוצמתי למשימות אד-הוק, אך לא אדריכלות ברירת מחדל. למסמכים אקדמיים, דוחות CTI ותהליכי ארגון — עבודה בטקסט עם תבנית CLS עדיפה, זולה וניתנת לבקרת איכות.

9 LaTeX, LuaLaTeX ויצירת PDF מקצועי

LaTeX הוא שפת סימון לייצור מסמכים מקצועיים [7]. במסגרת ההרצאה הוצג כלי מרכזי לעידן הסוכנים: מסמך שנכתב בטקסט בלבד, ללא גרירת עכבר, מתאים להפעלה אוטומטית על ידי AI.

9.1 מדוע LaTeX?

- **רפטיליות** — אותו קוד מייצר אותו PDF בכל הרצה.
 - **הפרדה** — תוכן (tex.) נפרד מעיצוב (cls.).
 - **מסמכים גדולים** — תוכן עניינים, ביבליוגרפיה, הפניות אוטומטיות.
 - **מתמטיקה** — נוסחאות ברמה ש-Word מתקשה להשיג.
 - **סוכן-ידידותי** — פקודות טקסט במקום קואורדינטות עכבר.
- המרצה הצהיר במפורש: "טעות פטאלית לעבוד בוורד" למסמכים רציניים וגדולים.

9.2 סוגי קבצי מסמך

- ההרצאה סקרה משפחת מסמכי טקסט:
- txt.** טקסט שטוח — ללא עיצוב, מתאים לקוד וסקילים.
 - md=** Markdown — תמונות, הדגשות, תמונות; פופולרי לתיעוד.
 - tex.** $\text{\LaTeX}$ — מסמכים אקדמיים ומקצועיים ל-PDF.
 - html.** דפי אינטרנט — ניתן להמיר ל- $\text{\LaTeX}$ על ידי סוכן.

9.3 LuaLaTeX — הקומפיילר

LuaLaTeX הוא מנוע קומפילציה מבוסס Lua, התומך ב-Unicode ובפונטים מודרניים (דרך fontspec) [8]. לעברית זה קריטי: pdfLaTeX אינו מספיק לשילוב עברית-אנגלית איכותי.

תהליך הקומפילציה:

1. כותבים main.tex + .cls + .bib.
2. מריצים lualatex main.tex.
3. מריצים bibtex main (אם יש ביבליוגרפיה).
4. מריצים lualatex פעמיים נוספות (תוכן עניינים והפניות).

Listing 5: דוגמת שימוש בפקודות התבנית

```

\template-ai-agentic\documentclass\ 1
\document\begin\ 2
\maketitlepage\ 3
\{RAG\}eng\-\{API\}eng\ 4
\{notebox\}begin\ 5
\{cls.\}texttt\ 6
\{notebox\}end\ 7
\document\end\ 8

```

9.4 מבנה פרויקט לסוכן

לפי דוגמת המרצה:

- `main.tex` — קובץ ראשי, יכול לכלול `\input{}`.
- `agentic-ai-template.cls` — כל העיצוב והפקודות המותאמות.
- `references.bib` — מקורות בפורמט BibTeX.
- `/sources` — חומר גלם (תמלול, סיכומים).

9.5 יתרונות לעבודה עם סוכן AI

תובנה

סוכן שכותב $\text{\LaTeX}$ מייצר diff קריא, ניתן לגרסה ב-Git, וניתן לתיקון נקודתי. מסמך Word שנוצר על ידי סוכן לעיתים "שובר" עיצוב בלתי נראה עד להדפסה.

9.6 מגבלות

- עקומת למידה — שגיאות קומפילציה דורשות הבנה בסיסית.
 - עברית-BiDi — בעיות RTL/LTR דורשות תבנית מותאמת.
 - התקנת חבילות — `TeX LiveMiKTeX` / נדרשים מקומית.
- פרקים 10–11 יראו כיצד CLS וסקילי QA פותרים חלק מהמגבלות הללו בצורה רפטבילית.

10 Reproducibility-CLS Templates

קובץ `cls` (document class) הוא לב העיצוב במסמכי $\text{\LaTeX}$. המרצה הציג אותו כפרויקט עצמאי שמבטיח שכל מסמך שייצור הסוכן ייראה אחיד — עקרון הרפטביליות (Reproducibility).

10.1 מהו קובץ CLS?

document class מגדיר:

- גודל עמוד, שוליים, פונטים (עברית ואנגלית).
 - עיצוב כותרות, כותרת עליונה ותחתונה.
 - פקודות מותאמות: `eng`, תיבות מידע, סגנון טבלאות.
 - כללים קבועים: קופירייט, מספור, צבעים ארגוניים.
- במקום שהסוכן יחזור על אותם הגדרות עיצוב בכל `main.tex`, הוא טוען:

```
\documentclass{template-ai-agentic}
```

10.2 CLS כ-Tool

המרצה תיאר את ה-CLS ככלי (tool): ניתן להגדיר בו פונקציות $\text{\LaTeX}$ חדשות — למשל נוסחאות במרכז שורה, בדיקת כיוון כתיבה, או הוספת כותרת עליונה אוטומטית. כל תיקון עיצובי חוזר נכנס ל-CLS ולא נשאר "תיקון חד-פעמי" במסמך.

10.3 לולאת תיקון רפטבילית

כאשר מתגלה באג ב-PDF (טבלה זזה ימינה, עברית הפוכה):

1. מציגים לסוכן את הבעיה — בלי לתקן מיד.
2. מבקשים הסבר והצעת תיקון (בקרת איכות).
3. אם הבעיה בשפה טבעית — מעדכנים Skill.
4. אם הבעיה טכנית-עיצובית — מעדכנים CLS.
5. מריצים קומפילציה מחדש; אם עבר — הבעיה נפתרה לכל המסמכים העתידיים.

אזהרה

תיקון נקודתי ב-main.tex במקום ב-clacls יגרום לבאג לחזור במסמך הבא. זהו ניגוד ישיר לרפטביליות.

טבלה 7: $\text{\LaTeX}$ / Word

מאפיין	LaTeX + CLS + Agent	Word + Agent
עקביות עיצוב גרסאות	גבוהה (תבנית) ברור Git diff	נמוכה קובץ בינארי
אוטומציה עברית מורכבת	קומפילציה מלאה LuaLaTeX דורש	לעיתים ידני בעיות נפוצות

10.4 יישום ארגוני

בארגון, מומלץ:

- תבנית CLS אחת לכל סוג מסמך (דוח, הצעת מחיר, מאמר).
- קצר לכל פרויקט — מגדיר מבנה וקהל יעד.
- סקריפט קומפילציה (compile.ps1) — אותה פקודה לכל אחד.

10.5 קשר למטלה 04

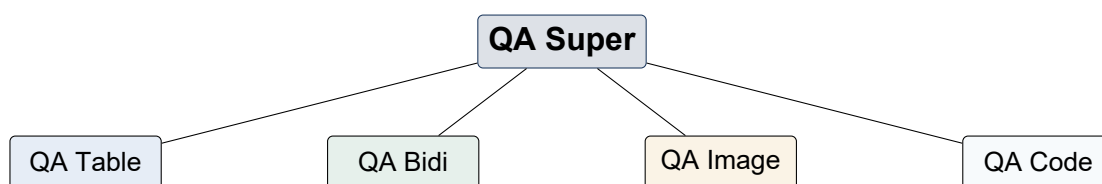
מטלה זו קובץ `agentic-ai-template.cls` אמיתי. ההערכה כוללת לא רק תוכן אלא גם את היכולת להפריד תוכן מעיצוב — מיומנות מרכזית בהגדרת הנחיות לסוכן.

תובנה

“הסוכן שלכם יהיה רפטבילי — הוא לא כל פעם ייצר משהו אחר” (תמלול ההרצאה). זהו עקרון ההנדסה שמאחורי מטלה 04.

11 QA Skills ובקרת איכות

ייצור PDF אוטומטי במסמכים ארוכים (30–50 עמודים) מייצר באגים חוזרים: טבלאות שבורות, עברית-אנגלית מעורבבת, תמונות חורגות מהשוליים. המרצה הציג מערכת QA Skills היררכית לטיפול בכך — מודל שניתן ליישם בכל פרויקט סוכן.



איור 6: מודל ה-QA Skills היררכי. מודל זה מפרק את QA לרמות שונות: QA Super, QA Table, QA Bidi, QA Image, QA Code.

11.1 פילוסופיית QA

כאשר מתגלה בעיה:

1. אל תתקן סתם — בקש הסבר מה קרה.
2. החלט: האם הפתרון שייך ל-Skill (שפה טבעית) או ל-CLS (טכני)?
3. עדכן את השכבה המתאימה.
4. הרץ מחדש ובדוק שהתיקון כללי.

11.2 מבנה היררכי

לפי דוגמת המרצה:

QA Super

“מנכ”ל” — מכיר את כל סוגי הבעיות, מפעיל מומחים.

QA Table

מחלקת טבלאות — יישור, עמודות, עברית-אנגלית בטבלה.

QA Bidi

כיווניות — RTL/LTR, מספרים, נוסחאות.

QA Image

תמונות — גבולות, כיתובים, מסגרות.

QA Code

קטעי קוד — רקע, גלישת שורות, התנגשויות.

כל “מחלקה” היא Skill (או סוכן עם Skill ייעודי) עם כללי זיהוי (detector) ותיקון (fixer).

Fixer-ו Detector 11.3

- **Detector** — רץ על PDF או על קוד מקור, מדווח בעיות (למשל: "הטבלה זזה ימינה").
- **Fixer** — מקבל דיווח, יודע לתקן (למשל: עדכון הגדרות עמודה ב-CLS).

Skill QA-Table דוגמת 11.4

Listing 6: קטע מ-QA Skill לטבלאות

```
1 Skill Table-QA #
2 : Detect
3 documents RTL in margin wrong to aligned Tables -
4 (headers Hebrew) reversed order Column -
5 : Fix
6 environment table inside {RTL}begin\ Apply -
7 RaggedLeft\ with columns {}p Use -
8 {table}AtBeginEnvironment@ cls.template-ai-agentic Update -
9 tex.main in table single the only patch :Never
```

QA יתרונות מערכת 11.5

טבלה 8: תוכן QA וטבלת PDF

רכיב	תפקיד	פלט
Detector	זיהוי חריגה	דוח בעיות
Fixer	תיקון ממוקד	ב-diff/SkillCLS
QA Super	תיאום	אישור לפני הגשה
Compile	אימות	סופי-PDF

יישום במטלה 04 11.6

רשימת בדיקות לפני הגשה:

- קומפילציה ללא שגיאות (compile.ps1).
- תוכן עניינים, רשימת איורים וטבלאות מעודכנים.
- ביבליוגרפיה עם ציטוטים [1], [2] בגוף המסמך.
- לפחות 30 עמודים, 5 טבלאות, 5 איורים.
- עברית-אנגלית ללא שבירות בולטות.

תובנה

QA Skills הם "בטיחות בדם נקנית" — כל שורה בקובץ ה-Skill מייצגת באג אמיתי מהעבר. עם הזמן הספרייה הופכת יקרת ערך.

12 Agent Orchestration: סוכן יחיד מול ריבוי סוכנים

כמשימות מתמערות, נדרשת החלטה ארכיטקטונית: סוכן אחד עם מגוון Skills ("קובעים") או מספר סוכנים במקביל, כל אחד עם התמחות. ההרצאה דנה ביתרונות, חסרונות והצורך ב-אורכוסטרטור.

12.1 סוכן יחיד, ריבוי קובעים

סוכן אחד בטרמינל יכול לטעון Skill שונה לכל משימה — היום מומחה לטבלאות, מחר מומחה לתמונות. היתרונות:

- פשטות ניהולית — מעקב אחרי תהליך אחד.
- עומס חישובי נמוך — רק instance אחד פעיל.
- קל לדבג — יודעים היכן קרתה שגיאה.

החיסרון: ביצוע **סדרתי**. אם בדיקת טבלאות לוקחת דקה ובדיקת תמונות דקה נוספת — סה"כ שתי דקות.

12.2 ריבוי סוכנים, קובע אחד לכל אחד

פתיחת מספר טרמינלים (או תהליכים) — כל סוכן עם Skill ייעודי. יתרון מרכזי: **מקביליות**. טבלאות בעמוד 2 ותמונות בעמוד 1 יכולים להיבדק בו-זמנית על אותו main.tex, כל עוד הם לא משנים את אותן שורות.

דוגמה מההרצאה: 50 משימות של דקה כל אחת —

- סוכן יחיד: 50 דקות.
- 50 סוכנים במקביל: כדקה אחת (בתיאוריה).

12.3 חסרונות ריבוי סוכנים

- מורכבות ניהולית — קשה לעקוב אחרי 50 תהליכים.
- סיכון לולאות וצריכת טוקנים לא מבוקרת.
- מגבלת חומרה — המחשב "נחנק" מעומס.
- התנגשויות כתיבה — נעול קבצים ברמת מערכת ההפעלה.

12.4 תפקיד האורכוסטרטור

- כשמפעילים מספר סוכנים, **חובה** סוכן-על (Orchestrator):
1. מחלק משימות לסוכני משנה.
 2. אוסף תוצאות ומרכיב מסמך שלם.
 3. מפקח על תקציב טוקנים וזמן.
 4. מטפל בהתנגשויות (תור כתיבה לקובץ).

הערה

במטלה 02 (תרגום רב-שלבי) יושמה אורכוסטרציה: שלושה סוכנים ברצף עם run_pipeline.py כמנהל. אותו עיקרון — עם אפשרות למקביליות בשלב ה-QA.

טבלה 9: תיאור תפקידי הסוכנים

ריבוי סוכנים	סוכן יחיד	מאפיין
מהירה (מקביל)	איטית	מהירות
מורכב	פשוט	ניהול
גבוהים	נמוכים	משאבים
קשה	קל	דיבוג
חובה	אופציונלי	צורך באורכוסטרטור

12.5 המלצה מעשית

- המרה ממליץ על **תמהיל**: לא 50 סוכנים בבת אחת, ולא סוכן יחיד לכל דבר. לדוגמה במטלה 04:
- סוכן תוכן — כותב פרקים לפי תמלול.
 - סוכן עיצוב — מתחזק CLS.
 - סוכן QA — מריץ קומפילציה ובודק PDF.

תובנה

שאלה קלאסית למבחן (לפי ההרצאה): זמן ביצוע 50 משימות של דקה — סוכן יחיד לעומת 50 סוכנים. הבנת המקביליות היא מפתח להטמעה ארגונית.

13 יישום בעולם הסייבר: CTI, OSINT וחקירת אירועי Ransomware

פרק זה מדגים כיצד עקרונות ארכיטקטורת הסוכן — MCP, Tools, SkillsRAG, ואורכוסטרציה — יכולים לתמוך ב-OSINT (CTICyber Threat Intelligence), (וחקירת אירועי Ransomware. הדגמה **מושגית**; אין כאן מערכת RAG חיה או סריקת אינטרנט אמיתית.

13.1 הקשר: Ransomware ו-CTI

Ransomware הוא תוכנת כופר שמצפינה נתונים ודורשת תשלום. חוקר איומים צריך לאסוף IOCs (Indicators of Compromise), לנתח הערות כופר, להצליב עם מאגרי MITRE ATT&CK [9] ולשתף מודיעין עם הצוות [10].

13.2 ארכיטקטורת סוכן לחקירה

1. **סוכן איסוף** — Skill ל-OSINT: קריאת דוחות, לוגים, הערות כופר (טקסט). כלים: קריאת קבצים, API למאגרי מידע מורשים.
2. **סוכן ניתוח** — RAG על מאגר פנימי: דוחות קודמים, IOCs היסטוריים, טכניקות ATT&CK.
3. **סוכן דיווח** — ייצור PDF מקצועי ב-L^AT_EX מתבנית CLS ארגונית (כמו במטלה 04).
4. **QA Super** — בודק עקביות טבלאות IOC, עברית-אנגלית בשמות דומיינים, וקישורים בביבליוגרפיה.

13.3 תרחיש לדוגמה

- אירוע:** זיהוי קובץ חשוד invoice_2026.exe והודעת כופר README_DECRYPT.txt.
הסוכן מבצע:
- חילוץ (hash (SHA-256), כתובות IP, דומיינים מהלוגים.
 - שליפה מ-RAG: "האם hash דומה הופיע בעבר?"
 - מיפוי לטכניקות T1486 (נתונים מוצפנים לכופר) ב-ATT&CK.
 - הפקת דוח PDF עם טבלת IOCs וציר זמן.

טבלה 10: Ransomware

סוכן	Skill	Tool / MCP	פלט
איסוף	OSINT-Collect	API, קריאת קבצים	IOC רשימת
ניתוח	CTI-Analyze	RAGMITRE ,	ממצאים
דיווח	Report-LaTeX	LuaLaTeX	PDF
QA	QA-Table	PDF בדיקת	אישור

13.4 דוגמת RAG ל-CTI

Listing 7: שליפת הקשר לחקירת כופר

```

1 "family ransomware ... abc123 SHA256" = query
2 (3=top_k , (query)embed)search.cti_vector_db = docs
3 entries MISP , reports incident past :include may docs #
4 (raw_iocs , docs)build_incident_summary = prompt
5 (prompt)generate.llm = report

```

13.5 שיקולי אבטחה ואתיקה

אזהרה

סוכן CTI חייב לפעול במסגרת מדיניות ארגונית: אין סריקת אינטרנט ללא הרשאה, אין שיתוף מידע רגיש בפרומפטים לענן ציבורי ללא אנונימיזציה, ורישום מלא (audit log) לכל פעולה.

13.6 מגבלות ההדגמה

- המאמר לא מפעיל מערכת אמיתית — הוא ממחיש עיצוב ארכיטקטוני.
- מקורות CTI דורשים אימות מול מאגרים מורשים (VirusTotalMISP, OSINT).
- OSINT חייב לעמוד בחוק ובתנאי שימוש של מקורות.

תובנה

העיקרון המחבר לקורס: כמו שמסמך אקדמי ב- $\text{\LaTeX}$ הוא רפטיבלי, כך דוח דוח CTI מתבנית CLS מאפשר לצוותים לשתף פורמט אחיד בכל אירוע — עם RAG על ידע היסטורי.

14 סיכום אינטגרטיבי: המשמעות הניהולית והטכנולוגית

פרק זה מחבר את הנושאים הטכניים למסגרת החלטות ארגונית: מה משתנה כשארגון מאמץ סוכני AI לא רק כצ'אט, אלא כשכבת ביצוע?

14.1 מעבר מ-GUI ל-API

המגמה המרכזית מההרצאה: תוכנות יחשפו יכולות בטקסט ו-API במקום להסתמך על GUI בלבד. ארגונית:

- צוותי פיתוח — עדיפות ל-API ו-MCP על פני אוטומציית מסך.
- צוותי תפעול — הגדרת Skills ותבניות מסמך (CLS).
- הנהלה — ציפייה לרפטיביליות ולא ל"קסם" חד-פעמי של סוכן.

14.2 שלוש שכבות בארגון

ניתן למפות את מודל השכבות לתפקידים:

Software

מערכות ליבה, ERP, מאגרי נתונים, קומפיילרים.

Skill

מפרטי עבודה: איך לכתוב דוח, איך לבדוק טבלה, איך לסכם פגישה.

Agent

נקודת כניסה למשתמש — מממש מטרות בשפה טבעית.

14.3 מודל עלויות

- טוקנים — חלון הקשר, צילומי מסך, היסטוריה ארוכה.
- זמן — סוכן יחיד לעומת מקביליות עם אורכוסטרטור.
- תחזוקה — Skills ו-CLS מקטינים עלות תיקון חוזר.

14.4 רפטביליות כיתרון תחרותי

ארגון שבנה:

- ספריית Skills מאושרת,
 - תבניות PDFCLS / אחידות,
 - מערכת QA היררכית,
- יכול לייצר עשרות דוחות, הצעות מחיר או מסמכי רגולציה באותה איכות — במרחק פרומפט. מתחרה שעדיין עורך Word ידנית נשאר מאחור.

14.5 סיכונים וניהול סיכונים

- הזיות מודל — במיוחד ב-CTI ובמשפט; נדרש אימות אנושי.
- דליפת מידע — פרומפטים עם נתונים רגישים.
- תלות בספק — מודל, MCP, ענן.
- ריבוי סוכנים ללא פיקוח — לולאות ועלויות.

טבלה 11: מודל תחזוקה וניהול סיכונים AI

שאלה	אם כן	אם לא
האם חוזר על עצמו?	CLSSkills +	GUI-תד-פעמי ב
האם דורש ידע פנימי?	מאובטחRAG	פרומפט כללי
האם מחובר לתוכנה?	APIMCP/	סוכן דפדפן
האם דורש מהירות?	ריבוי סוכנים + אורכוסטרטור	סוכן יחיד

14.6 קשר לתפקיד מומחה AI בארגון

המרצה ציפה שבוגרי הקורס יהיו "מובילי הטמעת AI" — לא רק משתמשי צ'אט, אלא מגדירי תבניות, Skills, תהליכי QA וארכיטקטורת אינטגרציה. מטלה 04 מאמנת בדיוק מיומנות זו: הגדרת PRD, בניית CLS, והפקת תוצר מקצועי.

תובנה

המעבר מטכנולוגיה לערך עסקי: סוכן שמייצר PDF אחד ב-30 שניות מחליף שעות עיצוב — אם התשתית (QACLS,) כבר קיימת.

15 סיכום ומסקנות

מסמך זה סקר את ארכיטקטורת סוכני AI כפי שנלמדה בקורס AI Agents, הרצאה 07, והרחיב אותה ליישום קצר בתחום הסייבר. להלן סיכום המסקנות העיקריות.

15.1 ממצאים מרכזיים

1. **שכבות** — סוכן בנוי על SkillSoftware, ו-Agent; הבחנה זו מנחה היכן למקם לוגיקה קבועה לעומת שפה טבעית.
2. **ידע** — LLM אינו זוכר; Context Window ו-RAG מספקים זיכרון עבודה ואחסון חיצוני.
3. **חיבוריות** — CLI, SDK, API, ו-MCP מאפשרים לסוכנים לפעול בתוכנות; MCP מציע עדכון דינמי לעומת Skill סטטי.
4. **ממשק** — סוכן GUI יקר ואיטי; עבודה טקסטואלית ו-LuaLaTeX עדיפה למסמכים רפטיבליים.
5. **איכות** — CLS וסקילי QA הופכים תיקונים לכלליים ולא חד-פעמיים.
6. **קנה מידה** — אורכוסטרציה מאזנת בין סוכן יחיד (פשטות) לריבוי סוכנים (מהירות).

15.2 תרומת מטלה 04

מטלה 04 מדגימה את כל העקרונות בפרויקט אחד:

- PRD.md — הגדרת דרישות לסוכן (מטא-מיומנות הקורס).
- agentic-ai-template.cls — רפטיביליות עיצובית.
- main.tex — תוכן מלא בעברית עם מונחי English.
- references.bib — ביבליוגרפיה בפורמט IEEE.
- compile.ps1 — תהליך בנייה שחוזר על עצמו.

15.3 כיווני המשך

- הרחבת ספריית QA Skills לפי באגים אמיתיים בקומפילציה.
- חיבור MCP למאגרי CTI מורשים בפרויקט גמר.
- ניסוי אורכוסטרציה: סוכן תוכן + סוכן CLS + סוכן QA במקביל.
- מעבר מתמלול שיעור ל-RAG פנימי לשאלות בקורס.

15.4 מילה אחרונה

עידן סוכני ה-AI מחייב לא רק פרומפטים טובים, אלא ארכיטקטורה: שכבות, ידע, כלים, תבניות ובקרת איכות. מי ששולט בכלי טקסט כמו $\text{\LaTeX}$ ובתבנית CLS יכול להפוך את הסוכן ממחולל טיטות למכונת הפקת מסמכים מקצועיים — בקורס, בארגון ובחקירות סייבר.

הערה

מקור עיקרי: sources/transcript-lecture07.txt. מקורות נוספים ברשימת הביבליוגרפיה — יש לאמת כתובות ופרטים לפני הגשה סופית.

מקורות

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *in Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2023.
- [5] OpenAI, "A practical guide to building agents," <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>, 2024, accessed: 2026-06-17.
- [6] Anthropic, "Model context protocol," <https://modelcontextprotocol.io>, 2024, accessed: 2026-06-17.
- [7] L. Lamport, *$\text{\LaTeX}$: A Document Preparation System*, 2nd ed. Addison-Wesley, 1994.
- [8] A. Reutenauer, E. Roux, and F. Charette, *Polyglossia: An alternative to babel for XeLaTeX and LuaLaTeX*, <https://ctan.org/pkg/polyglossia>, 2023, accessed: 2026-06-17.

MITRE ATT&CK, "MITRE ATT&CK framework," <https://attack.mitre.org>, 2024, [9]
.accessed: 2026-06-17

National Institute of Standards and Technology, "Guide to cyber threat [10]
information sharing," NIST, Tech. Rep. NIST SP 800-150, 2018. [Online].
<https://csrc.nist.gov/publications/detail/sp/800-150/final> :Available